

МИНИСТЕРСТВО ОБРАЗОВАНИЯ КУЗБАССА

ГПОУ «ЮРГИНСКИЙ ТЕХНОЛОГИЧЕСКИЙ КОЛЛЕДЖ»
ИМ. ПАВЛЮЧКОВА Г.А.
Отделение АИТ

СИСТЕМА ЗАПИСИ В АТЕЛЬЕ НА ПОШИВ И РЕМОНТ ОДЕЖДЫ

ОТЧЕТ ПО ПРАКТИЧЕКОЙ РАБОТЕ № 1

УП.01.01

ПРАКТИКА

Специальность

09.02.09 Веб-разработка

Выполнил студент гр. 454

Шило А.А.

Проверил преподаватель

Поликарпочкин М.В.

2026 г.

РАЗДЕЛ 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

В рамках выполнения практической работы разрабатывается база данных для системы онлайн-записи в ателье по пошиву и ремонту одежды. Данная предметная область выбрана в соответствии с вариантом 26 задания. Ателье предоставляет услуги по индивидуальному пошиву одежды из различных видов тканей, а также выполняет ремонт готовых изделий. Система должна позволять клиентам оформлять заказы, выбирать швею и отслеживать прогресс выполнения работы.

В ателье работают швеи, имеющие разные разряды квалификации. Каждый заказ характеризуется типом ткани (лён, хлопок, шёлк, джинса, вельвет и другие) и сложностью работы, которая оценивается по шкале от 1 (простая работа) до 5 (очень сложная работа). Для контроля качества и своевременного выполнения заказов ведётся журнал этапов, который включает раскрой, примерку, пошив и финишную обработку. На каждом этапе фиксируются дата и время начала и окончания работ. Дополнительным требованием системы является учёт фурнитуры – расходных материалов, используемых при пошиве и ремонте. К фурнитуре относятся пуговицы, молнии, нитки, кружево, ленты и другие материалы. Система должна контролировать остатки фурнитуры на складе и не допускать отрицательных значений.

Основные бизнес-правила системы

- Первое правило. Каждый заказ обязательно связан с одним клиентом и одной швейей. Нельзя создать заказ без указания клиента или швеи.
- Второе правило. Сложность заказа оценивается целым числом от 1 до 5. Недопустимы значения вне этого диапазона.
- Третье правило. Разряд швеи оценивается целым числом от 1 до 6. Недопустимы значения вне этого диапазона.

- Четвёртое правило. Заказ сложностью выше 3 (то есть 4 или 5) не может быть назначен швее с разрядом 2 или ниже. Это бизнес-правило обеспечивает качество выполнения сложных заказов.
- Пятое правило. Остатки фурнитуры на складе не могут быть отрицательными. При списании фурнитуры на заказ система должна контролировать наличие достаточного количества.
- Шестое правило. Каждый заказ проходит через четыре обязательных этапа в строгой последовательности: раскрой, примерка, пошив, финиш.
- Седьмое правило. Система должна выявлять заказы, которые находятся на одном этапе более трёх дней без перехода к следующему этапу, для своевременного вмешательства менеджера.
- Восьмое правило. Телефон и адрес электронной почты каждого клиента должны быть уникальными, чтобы избежать дублирования записей об одном клиенте.

РАЗДЕЛ 2. КОНЦЕПТУАЛЬНАЯ МОДЕЛЬ

На основе анализа предметной области были выделены следующие ключевые сущности и их атрибуты.

Сущность 1. Клиент (Client).

Клиент – это физическое лицо, обратившееся в ателье для получения услуг. Атрибуты сущности Client включают идентификатор клиента, который является первичным ключом; фамилию; имя; номер телефона; адрес электронной почты. Телефон и адрес электронной почты являются уникальными и обязательными для заполнения.

Сущность 2. Швея (Seamstress). Швея – это сотрудник ателье, выполняющий заказы. Атрибуты сущности Seamstress включают идентификатор швеи (первичный ключ), фамилию, имя и разряд квалификации. Разряд является целым числом в диапазоне от 1 до 6.

Сущность 3. Фурнитура (Accessory).

Фурнитура – это расходные материалы, используемые при пошиве и ремонте. Атрибуты сущности Accessory включают идентификатор фурнитуры (первичный ключ), наименование, остаток на складе и единицу измерения. Наименование является уникальным, остаток не может быть отрицательным.

Сущность 4. Заказ (GarmentOrder).

Заказ – это основная единица работы в ателье. Атрибуты сущности GarmentOrder включают идентификатор заказа (первичный ключ), ссылку на клиента, ссылку на швею, тип ткани, сложность работы, дату создания и статус выполнения. Сложность принимает значения от 1 до 5, статус может быть «в работе», «готов» или «отменён».

Сущность 5. Этап заказа (OrderStage).

Этап заказа – это запись о прохождении заказом определённой стадии производственного процесса. Атрибуты сущности OrderStage включают ссылку на заказ, наименование этапа, дату и время начала, дату и время окончания. Первичный ключ является составным и состоит из ссылки на заказ и наименования этапа.

Сущность 6. Использование фурнитуры (OrderAccessory).

Использование фурнитуры – это связующая сущность между заказом и фурнитурой. Атрибуты сущности OrderAccessory включают идентификатор записи (первичный ключ), ссылку на заказ, ссылку на фурнитуру и количество использованного материала.

Связи между сущностями.

Связь между клиентом и заказом является типом «один ко многим», так как один клиент может иметь много заказов. Связь между швейей и заказом также является типом «один ко многим», поскольку одна швея может выполнять много заказов. Связь между заказом и этапами заказа является типом «один ко многим» – один заказ имеет несколько этапов. Связь между заказом и фурнитурой является типом «многие ко многим», так как один заказ может использовать несколько видов фурнитуры, и один вид фурнитуры может использоваться в нескольких заказах. Эта связь реализуется через промежуточную таблицу OrderAccessory.

ER-диаграмма. На рисунке 1 представлена ER-диаграмма разработанной базы данных, выполненная в нотации Чена. Сущности изображены в виде прямоугольников, атрибуты – в виде овалов, связи – в виде ромбов. Кардинальность связей указана над линиями соединения. [На месте этого текста в отчёте размещается рисунок с ER-диаграммой, созданный в draw.io или другом инструменте.]

РАЗДЕЛ 3. ЛОГИЧЕСКАЯ МОДЕЛЬ И НОРМАЛИЗАЦИЯ

Реляционная схема базы данных. После преобразования концептуальной модели в реляционную схему были получены следующие таблицы.

Таблица «client» содержит столбцы client_id (целочисленный, автоинкрементируемый, первичный ключ), last_name (строковый, обязательный), first_name (строковый, обязательный), phone (строковый, обязательный, уникальный), email (строковый, обязательный, уникальный).

Таблица «seamstress» содержит столбцы seamstress_id (целочисленный, автоинкрементируемый, первичный ключ), last_name (строковый, обязательный), first_name (строковый, обязательный), skill_rank (целочисленный, обязательный, с проверкой в диапазоне от 1 до 6).

Таблица «accessory» содержит столбцы accessory_id (целочисленный, автоинкрементируемый, первичный ключ), name (строковый, обязательный, уникальный), stock (целочисленный, обязательный, по умолчанию 0, с проверкой неотрицательности), unit (строковый, обязательный).

Таблица «garment_order» содержит столбцы order_id (целочисленный, автоинкрементируемый, первичный ключ), client_id (целочисленный, обязательный, внешний ключ к таблице client), seamstress_id (целочисленный, обязательный, внешний ключ к таблице seamstress), fabric_type (строковый, обязательный), complexity (целочисленный, обязательный, с проверкой в диапазоне от 1 до 5), created_date (дата, обязательный, по умолчанию текущая дата), status (перечисление, по умолчанию «в работе»).

Таблица «order_stage» содержит столбцы order_id (целочисленный, обязательный, внешний ключ к таблице garment_order, часть составного первичного ключа), stage (перечисление, обязательный, часть составного первичного ключа), start_date (дата и время, обязательный, по умолчанию текущие дата и время), end_date (дата и время, может быть не заполнено).

Таблица «order_accessory» содержит столбцы order_accessory_id (целочисленный, автоинкрементируемый, первичный ключ), order_id (целочисленный, обязательный, внешний ключ к таблице garment_order), accessory_id (целочисленный, обязательный, внешний ключ к таблице accessory), quantity_used (целочисленный, обязательный, с проверкой положительности).

Обоснование нормализации. Проверка соответствия требованиям нормальных форм выполнялась последовательно.

Первая нормальная форма (1НФ) требует, чтобы каждый столбец содержал атомарные значения. В разработанной схеме это требование выполнено полностью. Телефон клиента хранится как одна строка, а не как список номеров. Тип ткани представлен одним строковым значением. Этапы заказа вынесены в отдельную таблицу, а не хранятся в виде списка в таблице заказов.

Вторая нормальная форма (2НФ) требует, чтобы все неключевые столбцы зависели от полного первичного ключа. Для таблиц с простым первичным ключом (client, seamstress, accessory, garment_order, order_accessory) это требование выполняется автоматически. Для таблицы order_stage первичный ключ является составным, но в этой таблице нет неключевых столбцов. Частичных зависимостей не обнаружено.

Третья нормальная форма (3НФ) требует отсутствия транзитивных зависимостей. В разработанной схеме все зависимости неключевых атрибутов идут только к первичному ключу. Разряд швеи хранится только в таблице seamstress, а не дублируется в таблице garment_order. Если бы разряд хранился в заказе, то при изменении разряда швеи пришлось бы обновлять все связанные заказы, что создало бы аномалию обновления.

Таким образом, разработанная реляционная схема полностью соответствует требованиям третьей нормальной формы.

РАЗДЕЛ 4. SQL-СКРИПТ СОЗДАНИЯ БАЗЫ ДАННЫХ

Ниже представлен полный листинг SQL-скрипта для создания базы данных, всех таблиц, ограничений, триггера и индексов. Скрипт выполняется в среде MySQL версии 8.0 и выше.

Ниже, на рисунке 1 , показан пример создания таблицы sql.

```
-- ТАБЛИЦА КЛИЕНТОВ

CREATE TABLE client (
    client_id INT AUTO_INCREMENT PRIMARY KEY,
    last_name VARCHAR(50) NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    phone VARCHAR(20) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE
);

-- ТАБЛИЦА ШВЕЙ

CREATE TABLE seamstress (
    seamstress_id INT AUTO_INCREMENT PRIMARY KEY,
    last_name VARCHAR(50) NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    skill_rank INT NOT NULL CHECK (skill_rank BETWEEN 1 AND 6)
);

-- ТАБЛИЦА ФУРНИТУРЫ

CREATE TABLE accessory (
    accessory_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    stock INT NOT NULL DEFAULT 0 CHECK (stock >= 0),
    unit VARCHAR(20) NOT NULL
);
```

Рисунок 1

РАЗДЕЛ 5. ТЕСТОВЫЕ ДАННЫЕ И ПРИМЕРЫ ЗАПРОСОВ

Заполнение базы данных тестовыми данными. Для проверки работоспособности базы данных были подготовлены тестовые данные. Ниже представлены INSERT-запросы для заполнения всех таблиц. Каждая таблица содержит от пяти записей. Пример показан ниже:

```
-- ЗАПОЛНЕНИЕ ТАБЛИЦЫ КЛИЕНТОВ

INSERT INTO client (last_name, first_name, phone, email) VALUES
('Иванова', 'Анна', '+79161234567', 'anna@mail.ru'),
('Петров', 'Игорь', '+79162345678', 'igor@mail.ru'),
('Сидорова', 'Мария', '+79163456789', 'maria@mail.ru'),
('Козлов', 'Дмитрий', '+79164567890', 'dmitry@mail.ru'),
('Новикова', 'Елена', '+79165678901', 'elena@mail.ru');

-- ЗАПОЛНЕНИЕ ТАБЛИЦЫ ШВЕЙ

INSERT INTO seamstress (last_name, first_name, skill_rank) VALUES
('Кузнецова', 'Елена', 4),
('Смирнова', 'Ольга', 2),
('Васильева', 'Татьяна', 5),
('Морозова', 'Ирина', 3),
('Фёдорова', 'Светлана', 1);

-- ЗАПОЛНЕНИЕ ТАБЛИЦЫ ФУРНИТУРЫ

INSERT INTO accessory (name, stock, unit) VALUES
('Пуговица белая', 200, 'шт'),
('Молния 50см', 50, 'шт'),
('Нитки чёрные', 30, 'катушка'),
('Кружево', 15, 'метр'),
('Лента атласная', 40, 'метр');
```

Примеры запросов.

Первый запрос использует соединение трёх таблиц. Бизнес-задача данного запроса заключается в получении сводной информации по каждому заказу: фамилия клиента, фамилия швеи, разряд швеи, тип ткани, сложность работы, наименование используемой фурнитуры и количество. Такой запрос позволяет менеджеру ателье видеть полную картину по каждому заказу. Смотреть рисунок 2.

```
SELECT
    o.order_id,
    c.last_name AS client_name,
    s.last_name AS seamstress_name,
    s.skill_rank,
    o.fabric_type,
    o.complexity,
    a.name AS accessory_name,
    oa.quantity_used
FROM garment_order o
JOIN client c ON o.client_id = c.client_id
JOIN seamstress s ON o.seamstress_id = s.seamstress_id
LEFT JOIN order_accessory oa ON o.order_id = oa.order_id
LEFT JOIN accessory a ON oa.accessory_id = a.accessory_id
ORDER BY o.order_id;
```

Рисунок 2

Второй запрос содержит группировку с агрегатной функцией и условием HAVING. Бизнес-задача запроса заключается в определении загруженности каждой швеи. Выводятся фамилия, имя, разряд швеи и общее количество заказов. Условие HAVING отбирает только тех швей, у которых есть хотя бы один заказ. Смотреть рисунок 3.

```
SELECT
    s.last_name,
    s.first_name,
    s.skill_rank,
    COUNT(o.order_id) AS total_orders
FROM seamstress s
LEFT JOIN garment_order o ON s.seamstress_id = o.seamstress_id
GROUP BY s.seamstress_id
HAVING total_orders > 0
ORDER BY total_orders DESC;
```

Рисунок 3

Третий запрос использует подзапрос для вычисления дат. Бизнес-задача запроса заключается в выявлении проблемных заказов, которые застряли на одном этапе более трёх дней без перехода к следующему этапу. Запрос выводит идентификатор заказа, фамилию клиента, название этапа, дату начала этапа и количество дней задержки. Пример на рисунке 4.

```
SELECT
    o.order_id,
    c.last_name AS client_name,
    os.stage,
    os.start_date,
    DATEDIFF(NOW(), os.start_date) AS days_stuck
FROM garment_order o
JOIN client c ON o.client_id = c.client_id
JOIN order_stage os ON o.order_id = os.order_id
WHERE os.end_date IS NULL
    AND os.start_date < DATE_SUB(NOW(), INTERVAL 3 DAY);
```

Рисунок 4

На рисунках 2.1, 3.1 и 4.1 представлены скриншоты результатов выполнения каждого из трёх запросов.

order_id	client_name	seamstress_name	skill_rank	fabric_type	complexity	accessory_name	quantity_used
1	Иванова	Кузнецова	4	шёлк	4	Пуговица белая	6
1	Иванова	Кузнецова	4	шёлк	4	Молния 50см	1
2	Петров	Смирнова	2	лён	2	Нитки чёрные	2
3	Сидорова	Васильева	5	джинса	5	Пуговица белая	4
4	Козлов	Морозова	3	хлопок	3	Кружево	3
5	Новикова	Фёдорова	1	вельвет	2	Лента атласная	2

Рисунок 2.1

last_name	first_name	skill_rank	total_orders
Кузнецова	Елена	4	1
Смирнова	Ольга	2	1
Васильева	Татьяна	5	1
Морозова	Ирина	3	1
Фёдорова	Светлана	1	1

Рисунок 3.2

order_id	client_name	stage	start_date	days_stuck
1	Иванова	примерка	2026-05-11 10:00:00	8
2	Петров	пошив	2026-05-14 09:00:00	5
3	Сидорова	раскрой	2026-05-14 09:00:00	5
4	Козлов	примерка	2026-05-16 10:00:00	3

Рисунок 4.2

РАЗДЕЛ 6. ПРОВЕРКА ОГРАНИЧЕНИЙ ЦЕЛОСТНОСТИ

Для демонстрации работы ограничений целостности были выполнены тестовые операции, нарушающие установленные правила.

Тест 1. Нарушение ограничения UNIQUE. Была выполнена попытка вставить клиента с уже существующим номером телефона.

```
INSERT INTO client (last_name, first_name, phone, email)
VALUES ('Тестов', 'Тест', '+79161234567', 'test@mail.ru');
```

В результате MySQL выдал следующее сообщение об ошибке:

```
INSERT INTO client (last_name, first_name, phone, email)
VALUES ('Тестов', 'Тест', '+79161234567', 'test@mail.ru')

Ответ MySQL: ?

#1062 - Дублирующаяся запись '+79161234567' по ключу 'client.phone'
```

Ограничение UNIQUE успешно предотвратило дублирование номера телефона.

Тест 2. Нарушение ограничения FOREIGN KEY. Была выполнена попытка создать заказ для несуществующего клиента с идентификатором 999.

```
INSERT INTO garment_order (client_id, seamstress_id, fabric_type, complexity)
VALUES (999, 1, 'лён', 2);
```

MySQL выдал сообщение об ошибке:

```
INSERT INTO garment_order (client_id, seamstress_id, fabric_type, complexity)
VALUES (999, 1, 'лён', 2)

Ответ MySQL: ?

#1452 - Cannot add or update a child row: a foreign key constraint fails (`r95122gp_1`.`garment_order`, CONSTRAINT `garment_order_ibfk_1` FOREIGN KEY (`client_id`) REFERENCES `client` (`client_id`) ON DELETE RESTRICT ON UPDATE CASCADE)
```

Ограничение FOREIGN KEY обеспечило ссылочную целостность, запретив создание заказа без существующего клиента.

Тест 3. Нарушение CHECK-ограничения.

Была выполнена попытка вставить фурнитуру с отрицательным остатком на складе.

```
INSERT INTO accessory (name, stock, unit) VALUES ('Тест', -5, 'шт');
```

MySQL выдал сообщение об ошибке:

```
INSERT INTO accessory (name, stock, unit) VALUES ('Тест', -5, 'шт')

Ответ MySQL: ?

#3819 - Check constraint 'accessory_chk_1' is violated.
```

Ограничение CHECK не допустило отрицательного значения остатка.

Тест 4. Нарушение бизнес-правила через триггер.

Была сделана попытка назначить заказ сложности 4 швее с разрядом 2.

```
INSERT INTO garment_order (client_id, seamstress_id, fabric_type, complexity)
VALUES (1, 2, 'шёлк', 4);
```

Триггер check_complexity_vs_skill_rank перехватил операцию вставки и выдал сообщение об ошибке:

```
INSERT INTO garment_order (client_id, seamstress_id, fabric_type, complexity)
VALUES (1, 2, 'шёлк', 4)

Ответ MySQL: ?

#1644 - Заказ сложности выше 3 нельзя назначить швее с разрядом 2
```

Таким образом, бизнес-правило успешно реализовано на уровне базы данных.

РАЗДЕЛ 7. ВЫВОДЫ

В ходе выполнения практической работы была спроектирована и реализована реляционная база данных для системы записи в ателье на пошив и ремонт одежды в соответствии с вариантом 26 задания.

Основные сложности возникли при реализации бизнес-правила о недопустимости назначения сложного заказа швее низкого разряда. Поскольку стандартное ограничение CHECK в MySQL не поддерживает подзапросы, данное правило было реализовано с помощью триггера BEFORE INSERT. Это продемонстрировало необходимость использования процедурных расширений SQL для реализации сложной логики целостности. Дополнительной сложностью стало использование зарезервированного слова RANK в качестве имени столбца, что было решено путём переименования столбца в skill_rank.

Схема базы данных соответствует требованиям третьей нормальной формы, что обеспечивает отсутствие аномалий вставки, обновления и удаления. Все первичные и внешние ключи настроены с продуманными действиями ON DELETE и ON UPDATE. Для каскадного удаления этапов при удалении заказа использовано ON DELETE CASCADE, а для защиты от удаления клиента или швеи, имеющих заказы, применено ON DELETE RESTRICT.

Три разработанных запроса демонстрируют различные аспекты работы с данными: соединение трёх таблиц для получения сводной информации, группировку с условием HAVING для анализа загруженности сотрудников и подзапрос с вычислением дат для выявления проблемных заказов. Все запросы успешно выполнены на тестовых данных.

Предложения по улучшению системы включают создание хранимой процедуры для автоматического обновления статуса заказа при завершении всех этапов, а также представление (VIEW) для упрощения первого запроса с множественными соединениями. Дополнительно можно добавить триггер для автоматического списания фурнитуры со склада при назначении её на заказ.

В результате выполнения работы приобретены следующие навыки: проектирование концептуальной и логической модели данных, нормализация до третьей нормальной формы, написание SQL-скриптов с ограничениями целостности, создание триггеров, составление сложных запросов с JOIN, GROUP BY, HAVING и подзапросами.

СПИСОК ЛИТЕРАТУРЫ

1. MySQL Official Documentation. MySQL 8.0 Reference Manual. – Oracle, 2025. – URL: <https://dev.mysql.com/doc/refman/8.0/en/>
2. Дейт К. Дж. Введение в системы баз данных. – 8-е изд. – М.: Вильямс, 2022. – 1328 с.
3. Голицына О.Л., Максимов Н.В., Попов И.И. Базы данных. – М.: Форум, 2021. – 400 с.
4. Connolly T., Begg C. Database Systems: A Practical Approach to Design, Implementation, and Management. – 7th ed. – Pearson, 2020. – 1440 p.
5. Грубер М. Понимание SQL. – М.: Лори, 2021. – 732 с.

Версия СУБД: MySQL 8.0.32

КОНТРОЛЬНЫЙ ВОПРОС №26

Идемпотентность SQL-скрипта означает, что его можно выполнить несколько раз подряд, и после каждого выполнения база данных будет находиться в одном и том же состоянии, а сам скрипт не будет завершаться ошибками из-за того, что какие-либо объекты уже существуют. При разработке скрипта `schema_variant26.sql` для базы данных ателье я предпринял следующие меры для обеспечения идемпотентности.

Первая мера: условное создание базы данных.

Вместо использования конструкции `DROP DATABASE ... CREATE DATABASE`, которая может быть запрещена на некоторых хостингах (как в моём случае на Beget), я применил конструкцию `CREATE DATABASE IF NOT EXISTS`. Это позволяет создать базу данных только в том случае, если она ещё не существует. При повторном выполнении скрипта эта команда просто игнорируется, не вызывая ошибки.

Вторая мера: переключение на нужную базу данных.

Команда `USE atelier_variant26` не вызывает ошибки, даже если скрипт выполняется повторно, так как база данных уже существует и выбрана.

Третья мера: удаление существующих таблиц перед созданием.

Перед созданием новых таблиц я выполняю их удаление с помощью `DROP TABLE IF EXISTS`.

Это гарантирует, что при повторном запуске скрипта старые таблицы с их данными и структурой будут удалены, а затем созданы заново. Важно соблюдать порядок удаления: сначала удаляются таблицы, имеющие внешние ключи на другие таблицы (дочерние), а затем таблицы, на которые ссылаются внешние ключи (родительские). В моём скрипте порядок удаления выглядит следующим образом.

Четвёртая мера: удаление существующего триггера.

Триггер `check_complexity_vs_skill_rank` также удаляется перед созданием с помощью `DROP TRIGGER IF EXISTS`. Это позволяет избежать ошибки при повторном создании триггера с тем же именем.

Пятая мера: использование конструкции `IF NOT EXISTS` при создании индексов.

Для индексов не требуется предварительного удаления, так как при создании таблицы заново индексы также создаются заново. Однако если бы индексы создавались отдельно от таблиц, следовало бы использовать `DROP INDEX IF EXISTS` перед созданием.

Почему выбран именно такой подход, а не полный `DROP DATABASE`?

В моём случае на хостинге Beget команда `DROP DATABASE` была отключена по соображениям безопасности. Поэтому я не мог использовать самый простой способ идемпотентности – полное удаление и пересоздание базы данных. Вместо этого я реализовал идемпотентность через удаление отдельных объектов внутри базы данных.

Что происходит при повторном выполнении скрипта?

При первом выполнении происходит следующее. Создаётся база данных, так как её ещё нет. Создаются все таблицы, триггер и индексы. Затем вставляются тестовые данные.

При повторном выполнении происходит следующее. База данных уже существует, поэтому команда `CREATE DATABASE IF NOT EXISTS` ничего не делает. Таблицы удаляются и создаются заново, при этом старые данные теряются. Триггер удаляется и создаётся заново. Тестовые данные вставляются снова. В результате база данных после каждого выполнения находится в идентичном состоянии, содержащем только тестовые данные из скрипта.

Ограничения такого подхода.

Главный недостаток описанного подхода заключается в том, что при повторном выполнении скрипта все ранее внесённые пользовательские данные теряются, так как таблицы удаляются. Поэтому в производственной среде идемпотентность достигается другими методами, например, использованием систем миграции, таких как Flyway или Liquibase, которые применяют только те изменения, которых ещё нет в базе данных. Однако для учебной практики, где требуется многократно пересоздавать структуру базы данных для тестирования и отладки, описанный подход является оптимальным.

Полный фрагмент скрипта, обеспечивающий идемпотентность.

```
-- Идемпотентное создание базы данных
CREATE DATABASE IF NOT EXISTS atelier_variant26
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE atelier_variant26;

-- Идемпотентное удаление объектов в правильном порядке
DROP TABLE IF EXISTS order_accessory;
DROP TABLE IF EXISTS order_stage;
DROP TABLE IF EXISTS garment_order;
DROP TABLE IF EXISTS accessory;
DROP TABLE IF EXISTS seamstress;
DROP TABLE IF EXISTS client;
DROP TRIGGER IF EXISTS check_complexity_vs_skill_rank;

-- Далее следуют CREATE TABLE, CREATE TRIGGER, CREATE INDEX
-- и INSERT-запросы для тестовых данных
```

Вывод

Применённые меры – CREATE DATABASE IF NOT EXISTS, DROP TABLE IF EXISTS в правильном порядке, DROP TRIGGER IF EXISTS – обеспечивают полную идемпотентность SQL-скрипта. Данный скрипт можно безопасно выполнять любое количество раз, и после каждого выполнения база данных будет содержать актуальную структуру и тестовые данные в соответствии с заданием. Ни одна из команд не вызовет ошибки при повторном выполнении, что и является главным требованием к идемпотентному скрипту.